Pimpri-Chinchwad Education Trust's
Pimpri Chinchwad College of Engineering
(An Autonomous Institute)
Affiliated to Savitribai Phule Pune University (SPPU)
ISO 21001:2018 Certified by TUV

Assignment 4: Smart Traffic Management for Emergency Vehicles Using Dijkstra's Algorithm

Name: Soham kulkarni	**PRN:** 124B1F056
Subject: Design and Analysis of Algorithms	**Batch:** TY - A3

GITHUB REPOSITORY LINK

https://github.com/SOHAM0007-CODER/DAA_ASSIGNMNET

Aim

To design and implement Dijkstra's single-source shortest path algorithm for a smart traffic management system that routes an ambulance from its current location to the nearest hospital in minimum travel time, to accommodate dynamic changes in road weights caused by traffic congestion, and to analyse its time complexity in comparison with other shortest path techniques.

Objectives

1. To model a city road network as a weighted graph of intersections and roads.
2. To apply Dijkstra's algorithm to obtain the shortest travel time from the ambulance's source intersection to all hospital nodes.
3. To handle dynamic weight updates arising from real-time traffic conditions.
4. To optimise the implementation using a priority queue so that it scales to thousands of intersections.
5. To compare Dijkstra's algorithm with other shortest path algorithms and to represent the optimal route visually.

Theory

Problem context. An ambulance must reach the nearest hospital in the least possible time. The city road network is modelled as a weighted graph G = (V, E), where every intersection is a vertex, every road is an edge, and the weight of an edge is the travel time in minutes along that road under current traffic. Since travel times are always positive, the network satisfies the non-negative weight condition required by Dijkstra's algorithm. The shortest route is the path of minimum total weight from the source S to a hospital node, not the path of fewest roads.

Greedy principle. Dijkstra's algorithm is greedy. It maintains a tentative distance dist[v] for every vertex, initialised to 0 for the source and to infinity for all others, and a set of finalised vertices. At each step it selects the unfinalised vertex u with the smallest tentative distance and marks it as finalised. Because all edge weights are non-negative, no later path can improve the distance of u, so the locally optimal choice is also globally optimal. This is the greedy-choice property that makes the algorithm correct.

Relaxation. After a vertex u is finalised, every outgoing edge (u, v) with weight w is relaxed: if dist[u] + w is smaller than dist[v], then dist[v] is updated to dist[u] + w and the parent of v is recorded as u. The parent array

holds the shortest path tree, and the actual route from S to a hospital is recovered by tracing parents backwards from the destination and reversing the sequence.

Priority queue optimisation. A naive implementation scans all vertices to find the minimum each time, costing $O(V^2)$, which is unacceptable for a city with thousands of intersections. Instead a min-heap keyed on tentative distance is used, so extracting the minimum costs O(log V) and each relaxation costs O(log V). A road network is sparse, since an intersection typically connects to four or five roads and E is of the order of V, so the heap based version is dramatically faster in practice. Lazy deletion is used: an improved distance is pushed as a new heap entry and stale entries are discarded when popped.

Dynamic weight updates. Traffic conditions change continuously, so edge weights are not static. Two strategies are used. In periodic recomputation the algorithm is re-run from the ambulance's current intersection whenever a traffic feed updates, which is simple and always correct. In incremental updating only the affected part of the shortest path tree is recomputed: if the weight of an edge decreases, that edge is relaxed and the change is propagated forward through the tree; if it increases, the sub-tree rooted at the affected vertex is invalidated and those vertices are re-relaxed from their remaining neighbours. Since the ambulance keeps moving, the source is reset to the next intersection on the route before each recomputation.

Visual representation. The network is drawn as a node-link diagram in which intersections are circles and roads are lines labelled with their current travel time. The finalised shortest path tree is shaded, and the chosen route from the source to the selected hospital is highlighted in a distinct colour with the cumulative time displayed at each intersection. This gives the driver a navigable view and lets the control room see the effect of congestion updates as the highlighted route shifts.

Algorithm

```
Dijkstra(G, S)
1.  for each vertex v in V
        dist[v] <- INFINITY ;  parent[v] <- NIL
2.  dist[S] <- 0
3.  PQ <- empty min-heap ;  insert (0, S) into PQ
4.  while PQ is not empty
        (d, u) <- ExtractMin(PQ)
        if d > dist[u] then continue        // stale entry
        for each edge (u, v) with weight w(u, v)
            if dist[u] + w(u, v) < dist[v]  // relaxation
                dist[v] <- dist[u] + w(u, v)
                parent[v] <- u
                insert (dist[v], v) into PQ
5.  return dist[], parent[]

ExtractRoute(parent, D)
1.  path <- empty list ;  v <- D
2.  while v != NIL :  append v to path ;  v <- parent[v]
3.  reverse path and return it
```

Complexity Analysis

Every vertex is extracted from the priority queue once, at a cost of O(log V), and every edge is relaxed at most once, each relaxation causing at most one insertion of cost O(log V). The total running time is therefore:

$$\mathbf{T(V, E) = O(V \log V) + O(E \log V) = O((V + E) \log V)}$$

For a sparse road network where $E \approx kV$ for a small constant k, this reduces to O(V log V), so a city of 10^5 intersections is processed in a few million operations rather than the 10^{10} required by the $O(V^2)$ form.

Implementation	Time Complexity	Remark
Simple array / linear scan	$O(V^2)$	Dense graphs, small V
Binary heap (priority queue)	**$O((V + E) \log V)$**	**Sparse road networks — preferred**
Fibonacci heap	$O(E + V \log V)$	Theoretically best, high constant
Auxiliary space	$O(V + E)$	Adjacency list + distance array

Because the algorithm computes distances to every vertex from a single source, the distances to all hospitals are obtained in one run, and the nearest hospital is simply the hospital node with the smallest final distance. Search can also be terminated early once the required destination is finalised.

Comparison with Other Shortest Path Algorithms

Algorithm	Time Complexity	Negative Edges	Source	Suitability for city road network
Breadth First Search	$O(V + E)$	No	Single	Only for unweighted graphs
Bellman–Ford	$O(V \cdot E)$	Yes	Single	Handles negative weights, far slower
Floyd–Warshall	$O(V^3)$	Yes	All pairs	Infeasible for thousands of nodes
A* Search	O(E) with heuristic	No	Single	Faster to one target, needs heuristic
Dijkstra (heap)	**$O((V + E) \log V)$**	**No**	**Single**	**Best fit for this problem**

Justification for choosing Dijkstra's algorithm. Travel times are always positive, so the restriction on negative weights does not apply and there is no reason to pay the $O(V \cdot E)$ cost of Bellman–Ford. Floyd–Warshall computes all-pairs distances at $O(V^3)$, which is far beyond feasible for thousands of intersections and wasteful when only one source matters. Breadth first search ignores edge weights and would return the route with the fewest roads rather than the fastest one. A* is faster towards a single fixed target but needs an admissible heuristic such as straight-line distance and does not give distances to every hospital in one pass. The heap based Dijkstra therefore offers the best balance of correctness, speed and simplicity for this application.

Conclusion

Dijkstra's algorithm was studied and applied to the problem of routing an emergency vehicle through a city road network. Modelling intersections as vertices and travel times as non-negative edge weights allows the greedy selection and relaxation steps to produce the minimum-time route to every hospital in a single execution, with the priority queue implementation running in $O((V + E) \log V)$ time and $O(V + E)$ space. Recomputation on traffic feed updates keeps the route valid under dynamic conditions, and the highlighted shortest path tree provides a clear visual for navigation. The comparison with Bellman–Ford, Floyd–Warshall, breadth first search and A* confirms Dijkstra's algorithm as the most suitable choice for a large, positively weighted urban road network.